Herbert Renner Silva Araujo - 588012

Relatório Técnico de Algoritmos de Ordenação

Este relatório técnico acompanha a submissão do código implementado para o estudo e comparação de cinco algoritmos de ordenação fundamentais vistos em sala, além de um algoritmo extra pesquisado e implementado. O objetivo é documentar as características dos algoritmos, apresentar a análise de desempenho através de tempos de execução e fornecer uma interpretação dos resultados observados.

1. Descrição dos Algoritmos de Ordenação Implementados

Foram implementados cinco algoritmos de ordenação. A seguir, são descritas suas principais características:

1.1. Bubble Sort

Característica	Detalhes
Complexidade de Pior Caso	$O(n^2)$
Estável	Sim
In Loco	Sim
Recursivo/Iterativo	Iterativo
Ideia Principal	Percorre repetidamente a lista, compara elementos adjacentes e os troca se estiverem na ordem errada. O processo se repete até que a lista esteja ordenada.

1.2. Selection Sort

Característica	Detalhes
Complexidade de Pior Caso	$O(n^2)$

Característica	Detalhes
Estável	Não
In Loco	Sim
Recursivo/Iterativo	Iterativo
Ideia Principal	Divide a lista em uma parte ordenada (esquerda) e uma desordenada (direita). Repetidamente, encontra o menor elemento na parte desordenada e o move para o final da parte ordenada.

1.3. Insertion Sort

Característica	Detalhes
Complexidade de Pior Caso	$O(n^2)$
Estável	Sim
In Loco	Sim
Recursivo/Iterativo	Iterativo
Ideia Principal	Constrói a lista ordenada, um item de cada vez. Percorre a lista e insere cada elemento na sua posição correta na sublista já ordenada à sua esquerda.

1.4. Merge Sort

Característica	Detalhes
Complexidade de Pior Caso	O{n log(n)} - (Independente como esta organizado a lista)
Estável	Sim
In Loco	Não
Recursivo/Iterativo	Recursivo

Característica	Detalhes
Ideia Principal	Algoritmo do tipo "Dividir Para Conquistar". Divide o array pela metade repetidamente até ter sub-arrays de um elemento. Em seguida, mescla (merge) as sub-arrays de forma ordenada.

1.5. Quick Sort

Característica	Detalhes
Complexidade de Pior Caso	$O(n^2)$ (em casos como listas já ordenadas ou quase ordenada, podendo chegar a O{n log n)}
Estável	Não
In Loco	Sim
Recursivo/Iterativo	Recursivo
Ideia Principal	Algoritmo do tipo "dividir para conquistar". Escolhe um elemento "pivô" e particiona o array em dois sub-arrays: elementos menores que o pivô e elementos maiores que o pivô. Em seguida, ordena recursivamente os sub-arrays.

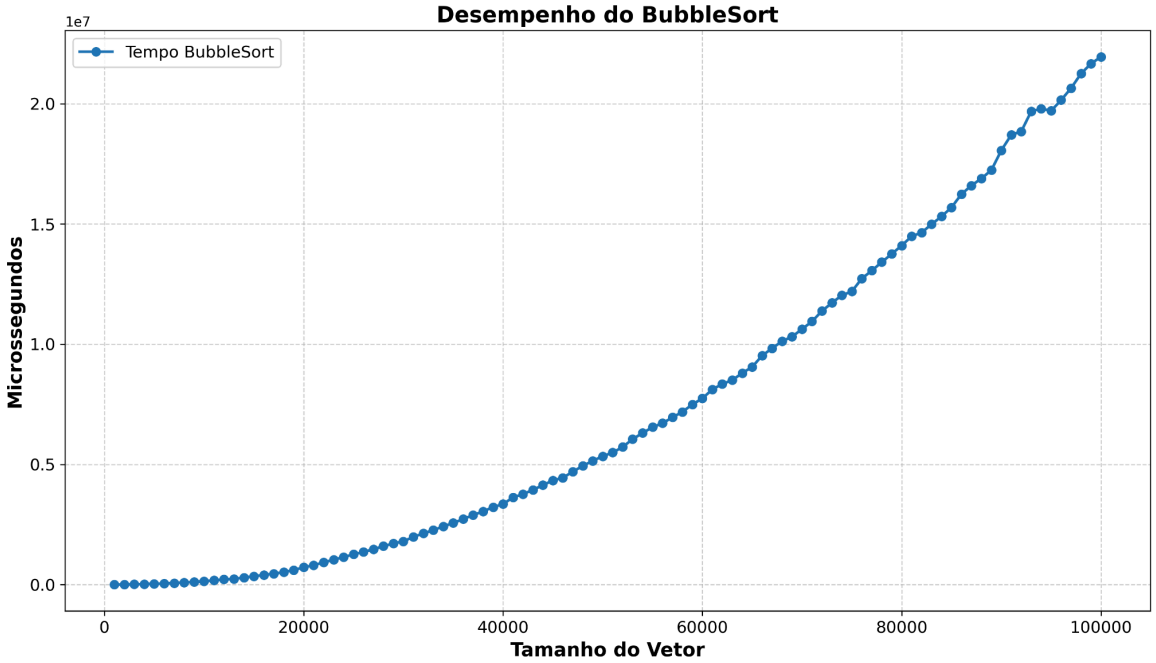
1.6. Bubble Sort Melhorado

Característica	Detalhes
Complexidade de Pior Caso	$O(n^2)$
Estável	Sim
In Loco	Sim
Recursivo/Iterativo	Iterativo
Ideia Principal	Percorre a lista como o Bubble Sort, mas inclui um *flag* para verificar se houve trocas

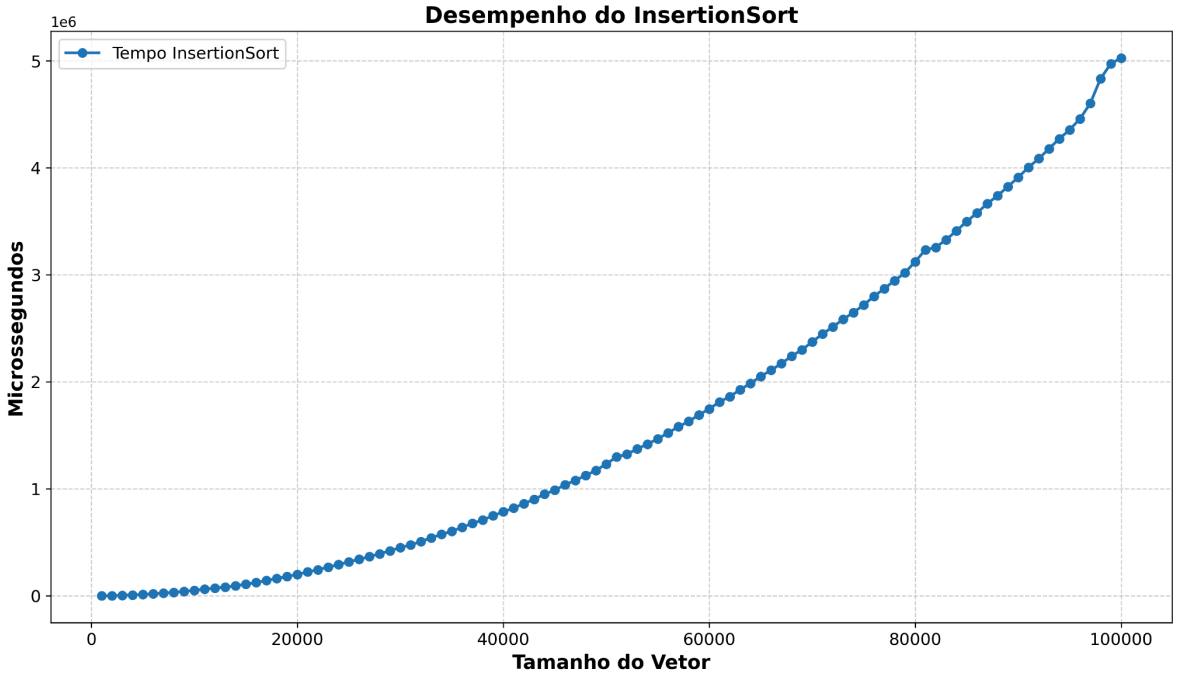
Característica	Detalhes
	em uma passagem completa. Se nenhuma troca for realizada, significa que a lista está ordenada e o algoritmo termina prematuramente, melhorando a performance no melhor caso.

2. Gráficos de Tempo de Execução para Distintos Tamanhos de Entrada

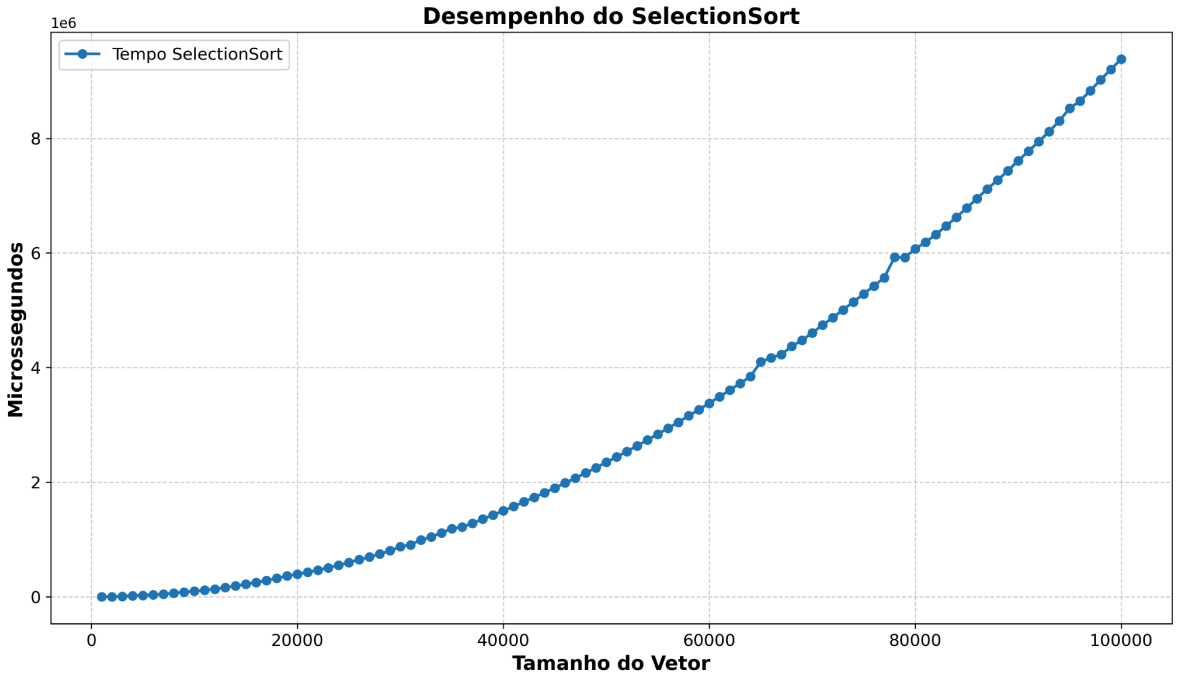
- Desempenho do **BubbleSorte** que para entradas pequenas se comporta de forma quase linear e para entradas grandes, se comporta semelhante a logatimica o que é valido dado a complexidade $O(n^2)$.



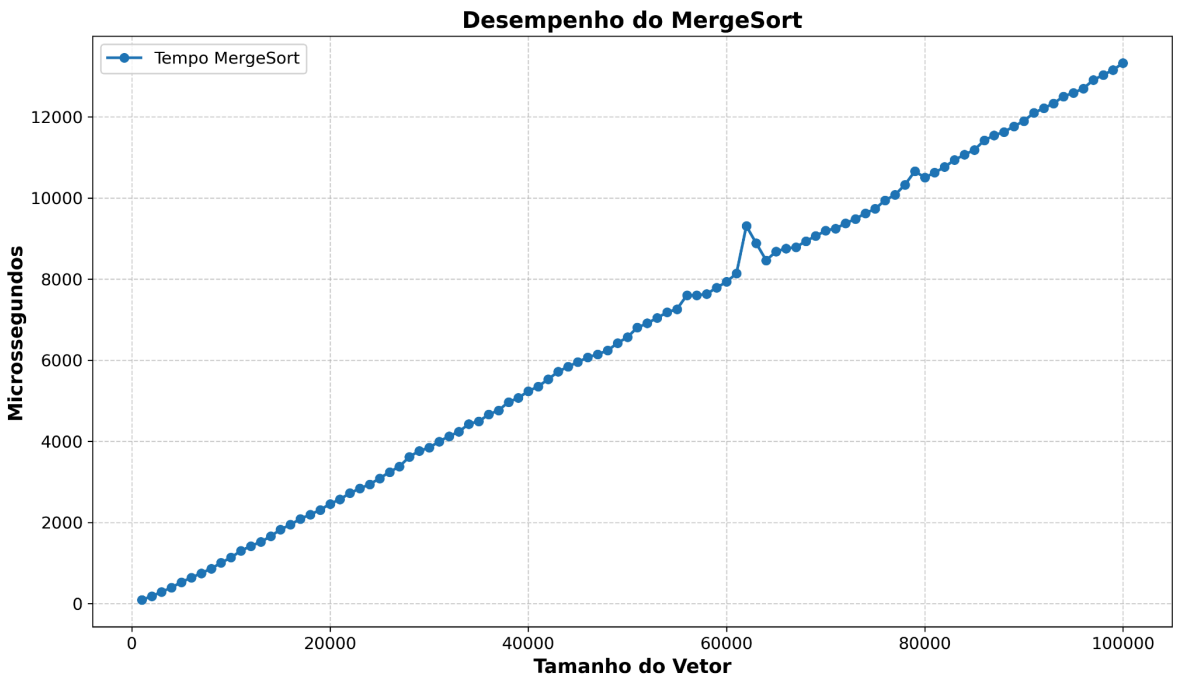
- Desempenho do **InsertionSort**: Para entradas pequenas se comporta de maneira quase linear e com o aumento do tamanos das entradas se comporta de forma semelhante a logaritimica, com pequenas oscilaçoes causada pelos vetores. O que é esperado para um algoritimo $O(n^2)$, mais mesmo essa complexidade ainda é melhor que o algoritimo bubblesort.



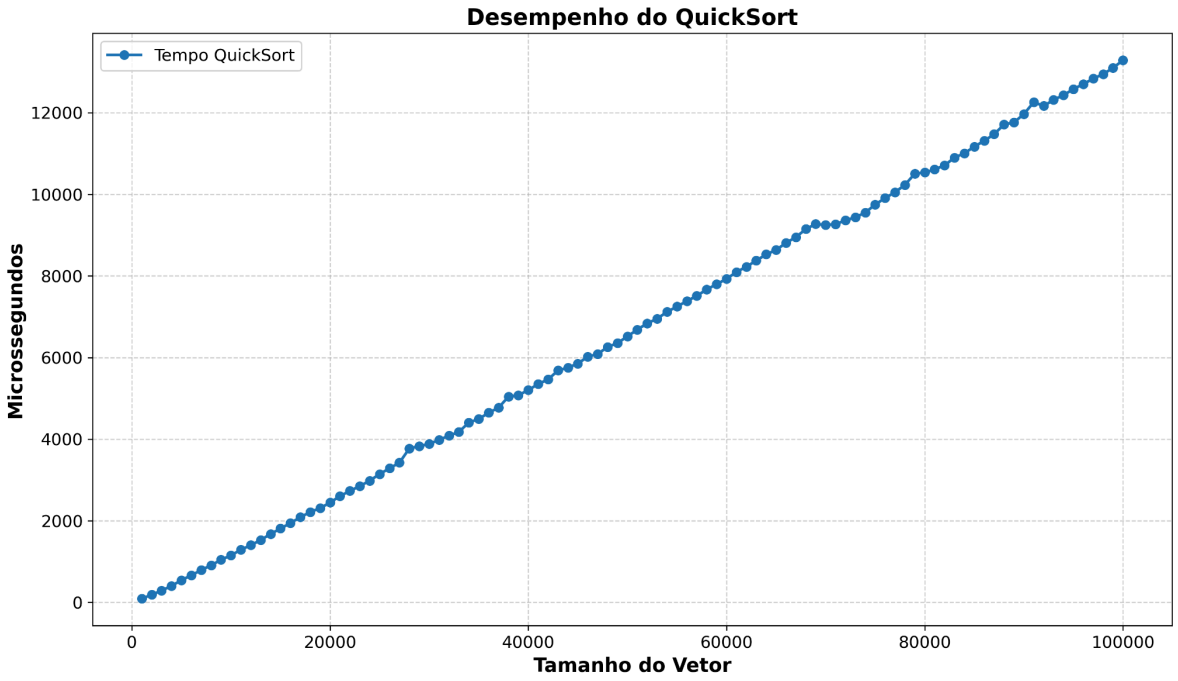
- Desempenho do **SelectionSort**: desempenho esperado para algoritimo de ordem O(n^2) porém pouco pior que o algoritimo insertionSort mesmo tendos mesma complexidade, com leves oscilaçoes causadas pelos vetores.



- Desempenho do **MergeSort**: desempenho esperado dado complexidade O(n log n) independente do vetor utilizado que se asemelha a uma reta, com uma pequena oscilação provavelmente causada por oscilaçoes da maquina.



- Desempenho do **QuickSort**: Desempenho esperado dado complexidade O(n log n) que se saiu muito semelhante ao MergeSort, só que com menos oscilaçoes.



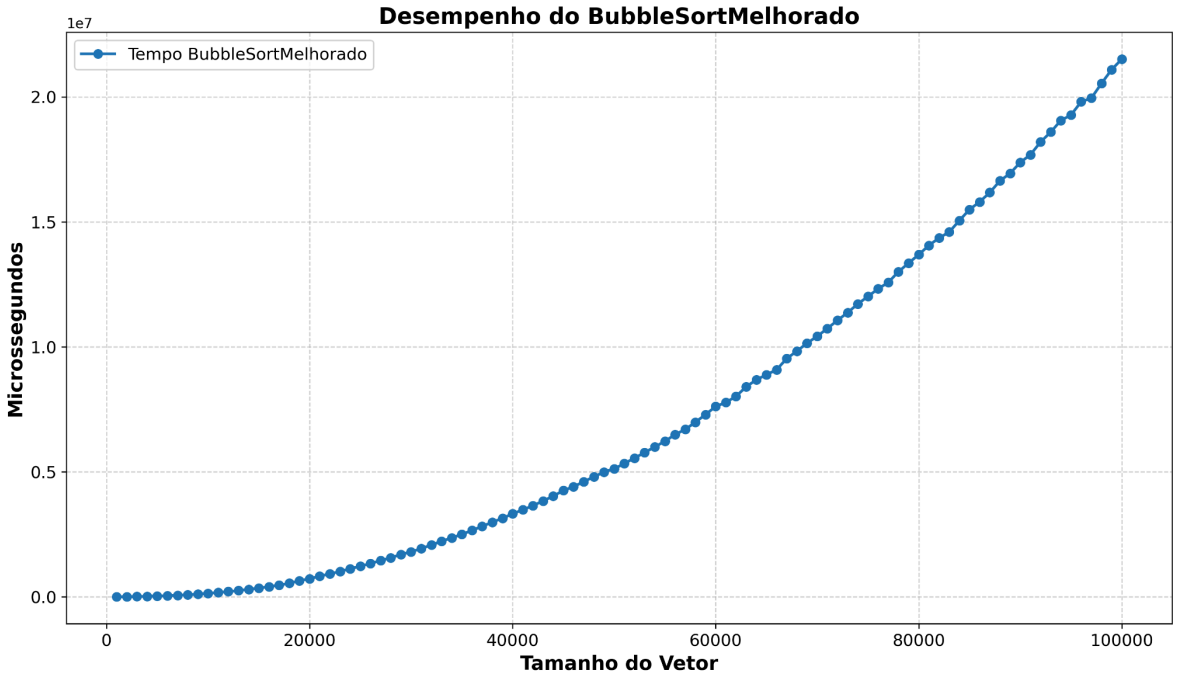
Configuração de maquina:

- **Hardware:** O ambiente de teste utilizou um processador **11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz**, com **8 GB de memória RAM DDR4**. Os testes

foram executados em um Sistema Operacional **Ubuntu 20.04 LTS**, garantindo um ambiente mais estável e com menor ruído de sistema para as medições.

3. Explicação do Algoritmo Pesquisado e Implementado

O algoritmo extra pesquisado e implementado foi o **Bubble Sort Melhorado**.



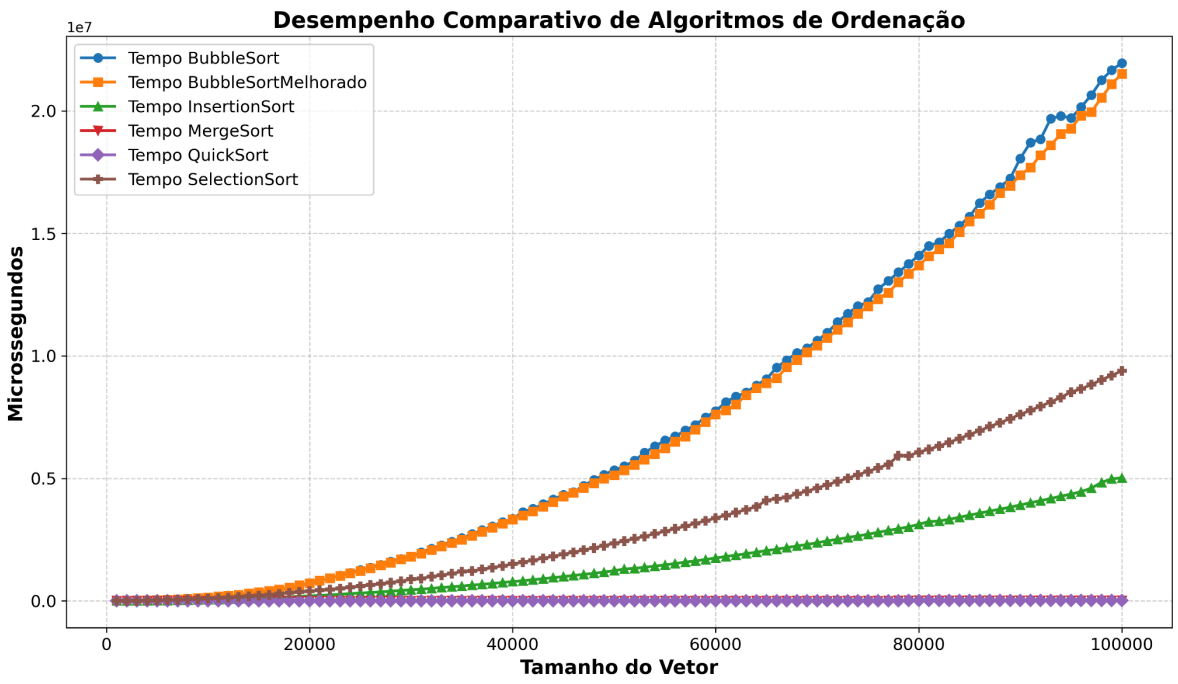
Ideia Principal: O algoritimo bubble sort melhorado é o mesmo algoritimo Bubble Sort, com a escesão de que ele elimina o pricipal problema desse algoritimo que é a verificação desnecesaria do vetro ja ordenado criando uma flag para a verificar se o vetor ja esta ordenado, se estiver o algoritimo encerra abruptamente evitando verificaçoes desnecesarias. Não da para notar diferença mais comparando os comparando juntos percebe-se.

Características:

- **Complexidade de Pior Caso:** $O(n^2)$, mesma complexidade do bubble tradicional.
- **Estável:** Sim.
- **In Loco:** Sim.
- **Recursivo/Iterativo:** Iterativo.

4. Comparação Gráfica dos Tempos de Execução

E aqui temos os seis graficos comparanto todos esses seis algoritimos:



Interpretação do Gráfico: Podemos perceber com maior clareza que a diferença entre algoritimo BubbleSortMelhorado e o tradicional é quse insignificante, dentro os algoritimos iterativos temos que o mais rapido é o InsertionSort e os algoritimos recursisvos se assemelha a complexidades constantes dado a sua rapidez em comparação a algoritimos de ordem $O(n^2)$.

5. Dificuldades Encontradas

Abaixo estão listadas as principais dificuldades encontradas durante a programação e elaboração deste relatório:

- **Tempo para colota de dados:** Mesmo com uma maquina rasoavel o tempo de execusão foi de aproximada aproximadamente 3-4 horas para a coleta de dados,pela necesidade de verificar os resultados.
- **Geração dos graficos**: Pouco conhecimento sobre a linguagem python e suas feramentas.
- **Problemas com sistema operacional**: falha na contagem de tempos dos algoritimos resultando tempos nulos, ocasionado por problemas com o (windows) necessitando a instação de outro sistema operacional (linux).

6. Bibliografia

As seguintes fontes foram consultadas para a pesquisa e implementação dos algoritmos:

- Slides de aula;
- https://youtube.com/playlist?list=PL9xmBV_5YoZOZSbGAXAPIq1BeUf4j20pl&si=-u8ZlrRiBocEI3xg;
- https://petbcc.ufscar.br/manualc/;